

ZK-RWA Allowlist

完整專案說明書 — 從零開始理解整個系統

HashKey Chain 「On-Chain Horizon」 Hackathon • ZKID Track • 2026 年 4 月

目錄

1. 先搞懂背景知識：RWA 是什麼？ERC-3643 是什麼？
2. 痛點：ERC-3643 有什麼致命缺陷？
3. 零知識證明（ZK Proof）60 秒速懂
4. Semaphore 協議：我們的 ZK 「黑盒子」
5. 我們的解法：ZK-RWA Allowlist 完整拆解
6. 系統架構與合約設計
7. 完整使用流程（附角色視角）
8. 隱私模型：什麼是私密的？什麼是公開的？
9. 真實世界應用場景
10. 與競品的差異分析
11. 為什麼這個痛點是真的？市場驗證
12. 技術細節深潛
13. 專案限制與未來發展
14. Demo 流程與評審要點

1. 先搞懂背景知識

1.1 RWA（Real-World Assets）是什麼？

RWA = 把現實世界的資產放到區塊鏈上，變成 token。舉例：

現實世界資產	代幣化 (tokenized) 後	好處
香港政府債券	HKGB30 token (我們 demo 用的)	24/7 交易、可分割、跨境流通
房地產基金份額	FUND-A token	小額投資、流動性提升
私募股權	PE-SERIES-B token	二級市場流通、結算秒完成

RWA 是目前區塊鏈產業最大的敘事之一。HashKey Chain 的整個定位就是「合規的 RWA 基礎設施」。這也是為什麼我們選這個方向 — **完美命中 HashKey 的核心商業策略**。

1.2 但 RWA 有個法律問題

不是任何人都能買賣這些 token。根據各國證券法規：

- **政府債券 token**：可能只有合格投資者 (accredited investor) 能持有
- **私募基金 token**：只有通過 KYC (身份驗證) 的人能買
- **合規要求**：發行方 (issuer) 必須確保每一筆轉帳的收發方都是經過核准的

核心需求：RWA token 必須有一個「白名單」(allowlist)，只有名單上的人能持有和交易 token。每一筆 transfer 都必須檢查：發送方和接收方都在白名單上。

1.3 ERC-3643 (T-REX)：目前的標準做法

ERC-3643 是以太坊上「合規 token」的產業標準，被稱為 T-REX (Token for Regulated EXchanges)。它的核心是一個 **鏈上身份註冊表 (Identity Registry)**：

```
// ERC-3643 的 Identity Registry (簡化版)
contract IdentityRegistry {
    // 公開的 mapping：任何人都能查
    mapping(address => bool) public isVerified;

    // 公開的事件：任何人都能掃
    event IdentityRegistered(address indexed investor);

    function registerIdentity(address investor) external onlyAgent {
```

```
        isVerified[investor] = true;          // 全世界都看得到  
        emit IdentityRegistered(investor);    // 全世界都掃得到  
    }  
}
```

Transfer 時，token 會檢查 `isVerified[from] && isVerified[to]` 。

2. 痛點：ERC-3643 的致命缺陷

ERC-3643 的身份註冊表是完全公開的。

任何人都可以呼叫 `isVerified(address)`，列舉出所有通過驗證的投資者地址。

這有什麼問題？想像一個情境：

情境：香港某家族辦公室用 ERC-3643 投資 RWA

1. 競爭對手前跑 (Front-running)

你的基金用 ERC-3643 發行了一個 tokenized 債券。競爭對手的分析師直接去鏈上掃描 `IdentityRegistered` 事件，10 秒鐘就拉出你所有 LP（有限合夥人）的地址。他們知道你的資金來源、LP 構成、甚至可以推測你的策略。

2. 記者曝光投資人名單

區塊鏈記者寫一個簡單的 script，就能列出所有持有某個 RWA token 的地址。配合鏈上分析工具（Arkham、Nansen），可以直接對應到真實身份。「某某家族辦公室持有 3000 萬美元 XX 債券 token」— 這種標題隔天就上新聞。

3. 社會工程攻擊

攻擊者知道你的地址在某個高價值 RWA 的白名單上。這代表你是高淨值個人。你立刻成為釣魚攻擊、SIM swap、甚至物理威脅的目標。

為什麼機構投資者在意這個？

傳統金融界有一條鐵律：「你的 LP 名單就是你的競爭護城河。」

在 TradFi（傳統金融），LP 名單是最高機密。基金管理公司不會公開自己的投資人是誰。券商不會公開客戶持倉。這是受法律保護的商業機密。

但 ERC-3643 把這個機密放到了一個全世界任何人都能讀取的公開帳本上。這不是理論上的風險 — 這是 RWA 領域目前最大的採用障礙之一。

ERC-3643 的世界

- 白名單完全公開
- 任何人都可以列舉所有投資者
- 鏈上分析可以追蹤持倉
- 機構投資者因此不敢用

我們想要的世界

- 白名單保持私密
- 合規檢查仍然在鏈上執行
- 非法轉帳仍然會被擋住
- 機構投資者可以安心使用

3. 零知識證明（ZK Proof）60 秒速懂

零知識證明讓你可以證明一件事是真的，但不需要透露任何細節。

生活中的比喻

場景：你要進一個 VIP 俱樂部，門口保安需要確認你在會員名單上。

傳統做法（= ERC-3643）	ZK 做法（= 我們的方案）
保安手上有一份完整的會員名單，貼在門口大家都看得到。 你報名字，保安對照名單✓。 但路過的人都能抄走整份名單。	保安手上只有一個密碼箱的鎖（= Merkle root）。 你用你的會員卡產生一個數學證明：「我確實能打開這個鎖。」 保安驗證你的證明✓。 但保安不知道你是名單上的第幾位，也看不到其他會員。

技術上，ZK proof 保證三件事

- 1. 完整性（Completeness）：如果你真的在名單上，你一定能生成有效的證明
- 2. 可靠性（Soundness）：如果你不在名單上，你造不出有效的證明
- 3. 零知識性（Zero-knowledge）：驗證者除了知道「你在名單上」之外，學不到任何其他資訊

在我們的專案裡，ZK proof 具體證明什麼？

持有者（holder）生成一個 proof，向合約證明：

- 1. 「我知道一個 identity commitment，它存在於 issuer 的 Merkle tree 裡。」
（= 我在名單上）

2. 「我控制的地址是 0xABC。」 (= 把 proof 綁定到我的錢包)
3. 「這個 proof 是針對 HKGB30 這個 token 的。」 (= scope 隔離)

合約驗證通過後，記錄 `admitted[HKGB30][0xABC] = true`。

但合約**完全不知道** 0xABC 對應的是名單上的哪一個 commitment。

4. Semaphore 協議：我們的 ZK「黑盒子」

4.1 為什麼用 Semaphore？

自己寫 ZK 電路（circuit）極度困難，需要數學碩士以上的密碼學知識。Semaphore 是 **Ethereum Foundation PSE 團隊** 維護的開源協議，已經過多次審計。我們把它當作黑盒子使用：輸入 → 魔法 → 輸出，不需要懂裡面的數學。

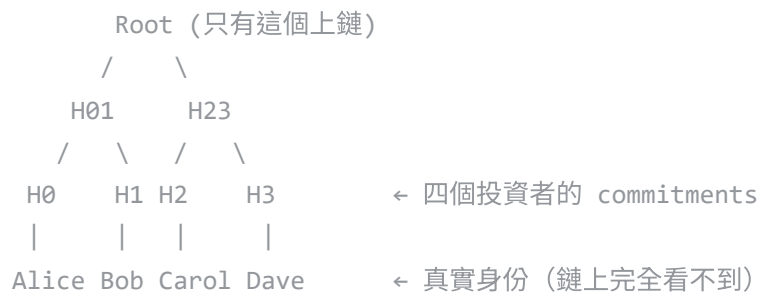
4.2 Semaphore 的四個核心概念

概念	是什麼	在我們專案裡的角色
Identity（身份）	一對隨機產生的密鑰（trapdoor + nullifier）。 從中衍生出一個 commitment（= 公鑰的 hash）	每個投資者在瀏覽器裡本地產生，存在 localStorage。 commitment 交給 issuer 加入群組。
Group（群組）	一棵 Merkle tree，葉子是 identity commitments	Issuer 在後端維護這棵樹。 只有 Merkle root（樹根）被發布到鏈上。 完整的葉子列表永遠不上鏈。
Proof（證明）	一個 Groth16 零知識證明，證明：「我的 commitment 是這棵 Merkle tree 的某個葉子」	投資者在瀏覽器裡用 WASM 生成（約 2-3 秒）。 然後提交到合約做鏈上驗證。
Nullifier（空值器）	一個 hash，用來防止同一個 identity 重複使用同一個 proof	每個 (identity, token) 組合產生唯一的 nullifier。 合約記錄已使用的 nullifier，防止重複 admit。

4.3 Merkle Tree 簡單說明

Merkle tree 是一種資料結構，讓你可以：

- 把 N 個資料 (commitment) 壓縮成一個固定大小的 **root** (32 bytes)
- 對任何一個資料，產生一個簡短的 **proof**，證明它確實是這棵樹的一部分
- 驗證者只需要看到 root + proof，**不需要看到完整的資料列表**



5. 我們的解法：ZK-RWA Allowlist 完整拆解

5.1 一句話描述

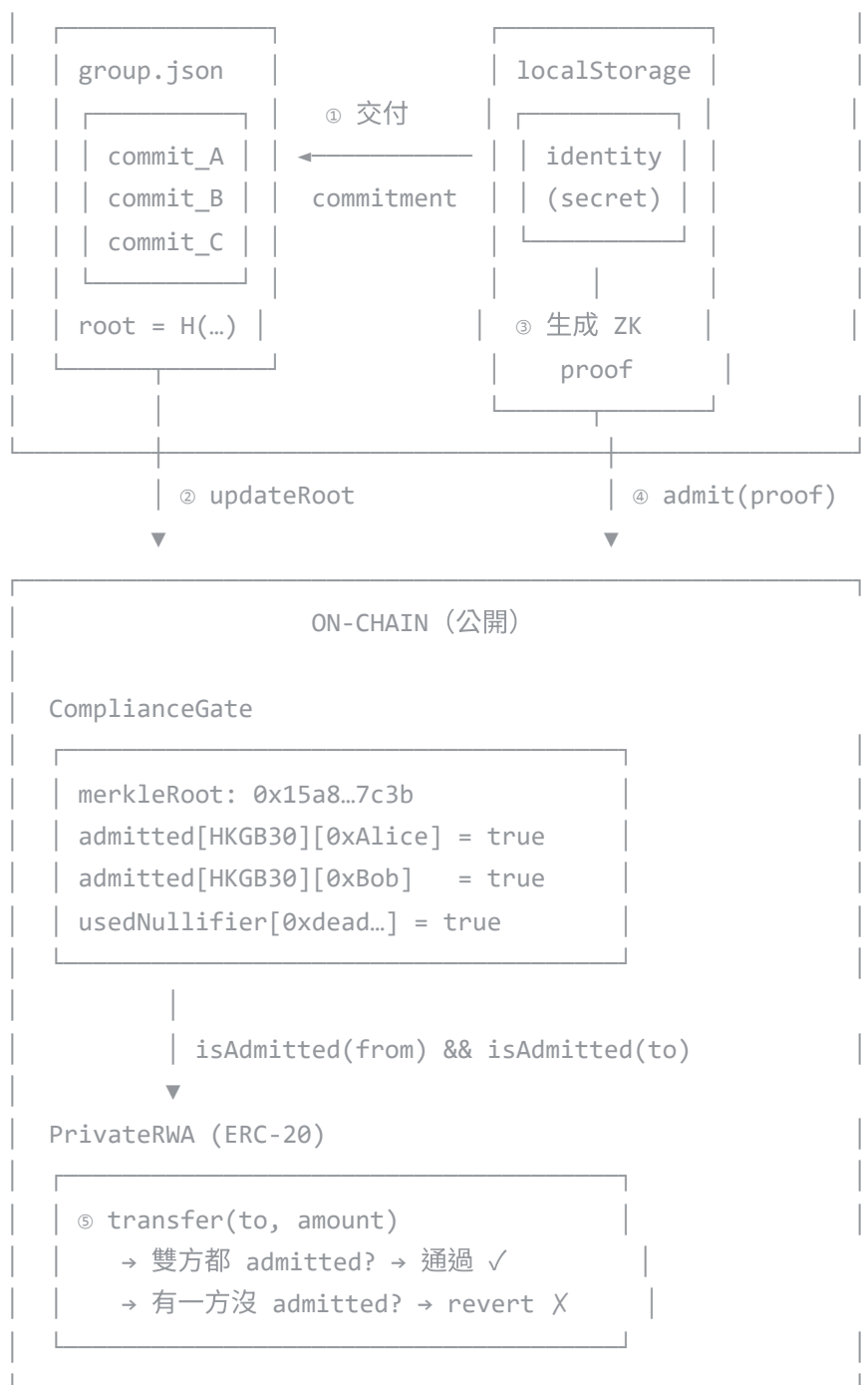
「Issuer 的投資者白名單以 Semaphore group 的形式存在 off-chain。鏈上只存 Merkle root。投資者自己用 ZK proof 證明自己在名單上，合約記錄他們的地址為 admitted。之後的轉帳就是普通 ERC-20，但 _update hook 會檢查雙方都是 admitted。」

5.2 三個角色

角色	是誰	在系統裡做什麼
Issuer	發行 RWA token 的機構 (例如：資產管理公司、政府、券商)	1. 維護 off-chain 的投資者群組 2. 把 Merkle root 發布到鏈上 3. Mint token 給已 admit 的投資者 4. 註冊 token 到 ComplianceGate
Holder	通過 KYC 的投資者 (例如：家族辦公室、合格投資者)	1. 在瀏覽器產生 identity 2. 把 commitment 交給 issuer 3. 用 ZK proof 自己 admit 到 token 4. 正常做 ERC-20 transfer
Public	鏈上的任何人 (例如：競爭對手、記者、攻擊者)	能看到：Merkle root、admitted 地址集合、token 餘額 看不到：完整名單、誰對應哪個 commitment、未 admit 的成員

5.3 整體流程（圖解）





6. 系統架構與合約設計

6.1 合約一：ComplianceGate.sol (~100 行)

這是系統的核心。它做三件事：

功能	函數	誰能呼叫	做什麼
更新 root	<code>updateRoot(newRoot)</code>	Issuer only	把新的 Merkle root 存到鏈上，保留最近 16 個歷史 root
自我准入	<code>admit(token, root, depth, nullifier, proof)</code>	任何人	驗證 ZK proof → 檢查 nullifier 沒用過 → 記錄 <code>admitted[token][msg.sender] = true</code>
查詢	<code>isAdmitted(token, holder)</code>	任何人	回傳 true/false，被 PrivateRWA 的 transfer hook 呼叫

為什麼保留 16 個歷史 root？

假設 Alice 在 12:00 產生了一個 proof，使用當時的 `root_v5`。但在 12:01 issuer 加了一個新成員，root 更新為 `root_v6`。Alice 在 12:02 才送出 tx — 如果只接受最新 root，她的 proof 會失敗。

保留最近 16 個 root 解決了這個 race condition。

6.2 合約二：PrivateRWA.sol (~40 行)

繼承 OpenZeppelin 的 ERC-20，唯一的修改是 override `_update` hook：

```
function _update(address from, address to, uint256 value) internal override {
    // mint (from == 0) 不檢查 sender
    if (from != address(0) && !gate.isAdmitted(address(this), from))
        revert NotAdmitted(from);

    // burn (to == 0) 不檢查 recipient
    if (to != address(0) && !gate.isAdmitted(address(this), to))
        revert NotAdmitted(to);

    super._update(from, to, value);
}
```

就這樣。不需要 ZK proof per transfer — admission 是一次性的。之後每次 transfer 就是一個簡單的 mapping lookup，gas cost 極低。

6.3 前端 / API 架構

頁面 / 路由	功能	技術
/join	Holder 產生 Semaphore identity	瀏覽器端，存 localStorage
/issuer	Issuer dashboard：管理群組、發布 root、mint	呼叫 API routes
/admit	Holder 產生 ZK proof + 送 admit tx	瀏覽器端 WASM proving
/wallet	查 balance、做 transfer	標準 ERC-20 interaction
/compare	ERC-3643 vs 我們的隱私對比圖	純靜態展示
/api/issuer/add-member	加入新 commitment 到 group.json	Node.js server
/api/issuer/publish-root	簽名 + 送 updateRoot tx	Node.js + viem
/api/group/root	回傳當前 group 狀態	讀 group.json

/api/group/merkle-path	回傳某 commitment 的 Merkle path	用於瀏覽器 proof gen
------------------------	------------------------------	-----------------

7. 完整使用流程（附角色視角）

Step 1：Holder 建立身份

Holder 開啟 /join → 點「Create identity」 →

瀏覽器產生 Semaphore identity → 存入 localStorage

Holder 看到的：一個很長的數字（identity commitment），例如：

9797040505547894238904504489...

外部看到的：什麼都沒有。沒有鏈上交易、沒有事件、完全 off-chain。

Step 2：Holder 把 commitment 交給 Issuer

Holder 複製 commitment → 透過安全管道交給 Issuer →

Issuer 在 dashboard 貼上

在真實世界，這會在 KYC 流程中發生。Issuer 確認投資者身份後，把 commitment 加入群組。在 demo 中，我們簡化為手動複製貼上。

Step 3：Issuer 加入成員並發布 root

Issuer 在 /issuer 點「Add」 → commitment 加入 group.json →

Merkle root 重新計算 → 點「Publish root」 → 鏈上 tx: updateRoot()

外部看到的：RootUpdated(0x15a8...7c3b) 事件。就只是一個 32 bytes 的 hash。完全無法反推出任何成員的資訊。

Step 4：Holder 產生 ZK proof 並自我 admit

Holder 開啟 /admit → 連接錢包 → 從 API 取得 Merkle path →

瀏覽器用 WASM 生成 ZK proof (~3秒) → 送出 admit() tx

外部看到的 : Admitted(HKGB30, 0xAlice, nullifier=0xdead...)

外部知道 0xAlice 是 HKGB30 的合規持有者了。但他們**不知道**：

- 0xAlice 對應 group 裡的哪一個 commitment
- group 裡還有多少其他成員（可能是 5 個、可能是 5000 個）
- 誰還沒有 admit（隱藏的成員完全隱形）

Step 5：正常使用 ERC-20

Alice 呼叫 transfer(Bob, 100) → _update 檢查 isAdmitted(Alice) ✓ →

_update 檢查 isAdmitted(Bob) ✓ → Transfer 成功 ✓

Alice 呼叫 transfer(Eve, 50) → _update 檢查 isAdmitted(Eve) X →

Revert: NotAdmitted(Eve)

8. 隱私模型：什麼是私密的？什麼是公開的？

資訊	狀態	原因
完整的投資者白名單	私密 ✓	只存在 issuer 的 group.json 裡，永遠不上鏈
哪個 commitment 對應 哪個地址	私密 ✓	ZK proof 的零知識性保證這個連結不會洩露
尚未 admit 的成員	私密 ✓	他們在鏈上完全不存在。沒有人知道他們是群組的一部分
群組的總人數	私密 ✓	Merkle root 不會洩露葉子的數量
已 admit 的地址集合	公開 (by design)	Transfer hook 需要檢查 <code>admitted[token][addr]</code> 如果這也是私密的，每次 transfer 都需要 ZK proof，UX 不可行
個別持有量 (token balance)	公開	標準 ERC-20 特性。要隱藏需要 shielded pool (不在 scope)
Merkle root	公開	必須公開，合約才能驗證 proof

關鍵 trade-off：我們提供的是「白名單級別的隱私」，不是「交易級別的隱私」。交易級別隱私需要 stealth addresses 或 shielded pool (如 Tornado Cash)，那是完全不同層級的系統。

但這正是機構真正需要的。機構投資者的核心需求不是隱藏單筆交易金額，而是「不要讓全世界知道我的 LP 名單」。我們精準地解決了這個痛點。

9. 真實世界應用場景

場景 A：Tokenized 政府債券

某國政府通過 HashKey Chain 發行 tokenized 債券。根據法規，只有持牌金融機構才能持有。用 ZK-RWA Allowlist：issuer（政府或其代理機構）維護一份合格機構名單。機構各自 admit 後即可交易。公眾只能看到「有 N 個地址持有此債券」，但不知道具體是哪些機構。

場景 B：私募基金 token

一個私募基金把 LP 份額 tokenized。LP 名單是基金最核心的商業機密。用 ERC-3643 的話，任何人都能查到 LP 是誰 — 競爭對手可以直接挖角。用 ZK-RWA Allowlist：LP 的身份在鏈上完全不可見。

場景 C：房地產 REITs token

一個 REITs（不動產投資信託）把份額 tokenized，只允許合格投資者持有。issuer 在完成 KYC 後把投資者加入群組。投資者 admit 後即可在二級市場交易。但無人能列出所有 REITs 投資者的清單。

10. 與競品的差異分析

ZKID Track 現有項目

項目	解決什麼問題	與我們的重疊度
FairDrop	Anti-sybil：一人只能 claim 一次 airdrop、投一次票	零重疊。面向社區/消費者場景，不是機構 RWA
Veritier	4 層合規身份（SFC/FATF mapped）+ AI agent identity	中度。他也做「合規」，但他做的是通用 KYC 分層，不是 RWA 白名單隱私。他的 tier 系統是公開的。
Web3ID	多鏈身份基礎設施平台（Root/SubIdentity）	低。範圍極廣但缺乏具體場景。3 分鐘 demo 很難講清楚。

為什麼我們會贏

維度	我們的優勢
痛點精準度	不是泛泛的「ZK identity」，而是精準解決一個具體、可量化的問題：「ERC-3643 會洩露投資者名單」。評審 3 秒就能懂。
HashKey 戰略吻合	HashKey = SFC 持牌 + RWA L2。我們的產品是他們 RWA 生態的缺失拼圖。HashKey Capital 的評審會立刻認出這個價值。
Demo 衝擊力	「轉帳給未 admit 的地址 → 交易被拒絕」這個畫面在 hackathon demo 中極其少見。大多數 ZK 專案只能展示「proof 驗證通過」，我們能展示一個實際的業務流程。
技術風險低	零自訂電路。Semaphore 已審計。合約 < 200 行。一個人 3.5 天可以做完。
標準化敘事	「ERC-3643 but private」是一個很強的 pitch 框架。指出現有標準的具體缺陷 + 提出具體替代方案 = 強故事性。

11. 為什麼這個痛點是真的？

11.1 ERC-3643 的實際採用情況

- ERC-3643 已被 Tokeny Solutions 用於超過 \$28B 的資產 tokenization
- 被部署在 Ethereum、Polygon、Avalanche 等多條鏈上
- 主要客戶：Societe Generale（法國興業銀行）、BNP Paribas 等大型金融機構

11.2 隱私問題已被提出

- 多份 RWA 產業報告指出「投資者隱私」是機構採用鏈上 RWA 的主要阻礙之一
- SEC（美國證監會）和 SFC（香港證監會）都要求保護投資者資訊隱私
- GDPR（歐盟資料保護法規）嚴格限制個人資料的公開處理 — 公鏈上的公開名單直接違反 GDPR

11.3 目前沒有標準解法

在我們之前，如果你想要一個合規的 RWA token 且保護投資者隱私，你的選擇是：

1. **用中心化系統** — 不放鏈上，失去區塊鏈的所有好處
2. **用 ERC-3643** — 合規但洩露投資者名單
3. **用 permissioned chain** — 完全私有鏈，失去互操作性

我們提供了第四個選項：**公鏈上的合規 RWA，但白名單是私密的**。這是一個之前不存在的解法。

12. 技術細節深潛

12.1 ZK 證明生成流程（瀏覽器端）

```

瀏覽器呼叫 generateProof(identity, merkleProof, message, scope)
|
| message = BigInt(holderAddress)  ← 綁定 proof 到錢包地址
| scope   = BigInt(tokenAddress)   ← 綁定 proof 到 token (nullifier 隔離)
|
| snarkjs 載入 .wasm + .zkey artifacts (從 CDN 下載 + 快取)
| 內部: Poseidon hash 計算 nullifier = H(scope, identity.nullifier)
| 內部: Poseidon hash 計算 messageHash = H(message)
| 內部: 驗證 identity.commitment 在 Merkle tree 中
| 內部: Groth16 證明產生
|
└ 輸出 SemaphoreProof:
    | merkleTreeRoot: string  (Merkle root, 合約會比對)
    | merkleTreeDepth: number (樹的深度)
    | nullifier: string       (唯一值, 防重複 admit)
    | message: string         (holder address hash)
    | scope: string           (token address hash)
    └ points: string[8]       (Groth16 proof 的 8 個點)

```

12.2 鏈上驗證流程

```

ComplianceGate.admit(token, root, depth, nullifier, points) {
|
| 檢查 token 已註冊 ✓
| 檢查 depth 在 [1, 32] 範圍 ✓
| 檢查 root 是已知的 (最近 16 個) ✓
| 檢查 nullifier 沒用過 ✓
|
| 計算 messageHash = keccak256(msg.sender) >> 8
|   └ 這確保 proof 是用「我的地址」生成的
|       別人不能拿我的 proof 去 admit 他們自己
|
| 計算 scopeHash = keccak256(tokenAddress) >> 8
|

```

```
└─ 呼叫 SemaphoreVerifier.verifyProof(  
  │   pA, pB, pC,  
  │   [root, nullifier, messageHash, scopeHash],  
  │   depth  
  │ )  
  └─ 這是 bn254 pairing 運算 (EIP-196/197 precompiles)  
      驗證 Groth16 零知識證明的正確性  
  
└─ 標記 nullifier 已使用 : usedNullifier[nullifier] = true  
└─ 標記 admitted : admitted[token][msg.sender] = true  
└─ 發出事件 : Admitted(token, msg.sender, nullifier)  
}
```

12.3 安全性保證

攻擊向量	防禦機制
重放攻擊：用同一個 proof admit 兩次	nullifier 記錄。同一個 (identity, scope) 只能產生一個 nullifier，第二次會 revert
冒名攻擊：A 拿 B 的 proof 去 admit	proof 的 message = msg.sender。合約重新計算 messageHash 並與 proof 比對
偽造 proof：不在群組裡的人造假 proof	Groth16 的可靠性保證。在數學上不可能偽造通過驗證的 proof
過期 root：issuer 更新了 root，舊 proof 失效	保留 16 個歷史 root。只要 root 在歷史中就接受
Cross-token 攻擊：用 token A 的 admit 去 token B	scope = tokenAddress。每個 token 產生不同的 nullifier

12.4 技術棧一覽

層	技術	版本
ZK 證明系統	Semaphore Protocol (Groth16 + bn254)	v4.14.2
智能合約	Solidity + Foundry + OpenZeppelin	0.8.24 / 1.5.1 / 5.6.1

前端	Next.js 14 App Router + TypeScript	14.2
鏈互動	viem (直接 window.ethereum)	2.x
目標鏈	HashKey Chain Testnet	chainId 133

13. 專案限制與未來發展

13.1 明確的 scope cuts（已知限制）

限制	原因	未來如何解決
沒有交易級隱私	需要 stealth address 或 shielded pool，3.5 天做不完	整合 Railgun 或類似方案
單一 issuer（硬編碼）	Hackathon scope。多 issuer 需要 access control 重構	改為 role-based，支持多個 issuer
沒有身份撤銷（revocation）	Semaphore v4 的 group removal 需要 Merkle proof 更新	issuer 停止發布包含該成員的 root
沒有真實 KYC 整合	Demo 環境。真實場景需要對接 KYC provider	串接 Sumsub / Onfido API
Group 存在 JSON file	Hackathon 快速開發。不適合生產環境	改用 PostgreSQL + 加密存儲

13.2 這個方案的固有限制（無法簡單解決的）

- 1. **Admitted 地址是公開的。**一旦你 admit 了，你的地址就公開可查。這是設計上的 trade-off — 如果 admitted set 也是私密的，每次 transfer 都需要 ZK proof，UX 會極差（每次轉帳等 3 秒做 proof + 額外 gas）。
- 2. **Identity 綁定瀏覽器。**如果用戶清除 localStorage 或換電腦，identity 就丟了。需要 issuer 重新加入新 commitment。
- 3. **Issuer 是中心化的信任點。**Issuer 知道完整的成員名單（因為他維護 group）。如果 issuer 被駭客攻擊，名單可能洩露。但這跟傳統金融的模型一致 — issuer 本來就需要知道投資者身份。

14. Demo 流程與評審要點

14.1 三分鐘 Demo 腳本

1 0:00 – 0:20 痛點引入

「ERC-3643 是目前 RWA token 的合規標準，但它有一個致命缺陷：投資者白名單是完全公開的。任何人都能列舉你的所有投資者。」 → 展示 /compare 頁面的對比。

2 0:20 – 0:40 解法概述

「ZK-RWA Allowlist 用 Semaphore 零知識證明把白名單移到 off-chain。鏈上只有 Merkle root，投資者用 ZK proof 自我 admit。」 → 展示架構圖。

3 0:40 – 1:00 Issuer 視角

→ 展示 /issuer dashboard：已有 3 個成員，root 已發布。 → 加入一個新成員（貼 commitment） → 發布新 root → 鏈上 tx。

4 1:00 – 1:40 Holder 視角（核心環節）

→ /join 產生 identity → 複製 commitment（之前已由 issuer 加入） → /admit → 連接錢包 → 「Generate proof」 → WASM spinner (~3秒) → 送出 tx → 鏈上顯示 Admitted 事件 → 「注意：事件裡只有 nullifier，沒有任何可以追溯到 commitment 的資訊」

5 1:40 – 2:10 Token 操作

→ Issuer mint 100 HKGB30 到 holder → /wallet 顯示 balance → Transfer 到另一個已 admit 的地址 → 成功 ✓

6 2:10 – 2:40 失敗展示（demo 的高潮）

→ 嘗試 transfer 到一個未 admit 的地址 → 交易被拒絕！ NotAdmitted(0x...) → 「這就是鏈上合規 — 不是靠 off-chain monitor，是合約直接 revert。」

7 2:40 – 3:00 收尾

→ 回到 /compare 頁面 → 「ERC-3643 洩露你的完整投資者名單。ZK-RWA Allowlist 解決了這個問題。零自訂電路、200 行合約、Semaphore v4 審計通過。Built for HashKey

Chain.」

14.2 評審會看什麼

評審維度	我們的表現
創新性	唯一做 RWA 白名單隱私的。直接指出 ERC-3643 的缺陷。
技術完成度	合約已部署、前端可用、proof 在瀏覽器跑、E2E 可 demo
與 HashKey 的契合度	HashKey = 合規 RWA L2。我們是他們 RWA 生態的缺失拼圖。
商業可行性	真實痛點（ERC-3643 隱私問題）+ 真實市場（\$28B+ RWA tokenization）
Demo 品質	失敗轉帳 = 即時可見的合規執行。很少 hackathon 能做到這種 demo。